2016 0822

C语言

- 前言
  - 计算机语言
    1. 程序是用特殊的编程语言写出来表达如何解决问题的。
    2. 不是用编程语言来和计算机交谈，而是描述要求它如何做事情的过程/方法

算法：计算机所有的事情都叫做计算。计算的步骤都叫做算法。

- 程序的执行
  - 解释：借助一个程序，那个程序能试图理解你的程序，然后按照你的要求执行
  - 编译：借助一个程序，就像一个翻译，把你的程序翻译成计算机真正能懂的语言——机器语言——写的程序，然后，这个机器语言写的程序就能直接执行了

前二

解释语言 Vs 编译语言

- 语言本无编译/解释之分
- 常用的执行方式而已
- 解释型语言有特殊的计算能力
- 编译型语言有确定的运算性能

- C语言在tiobe排行榜上常年稳居
- 现代的编程语言在语法上的差别小
- 语言的能力/适用领域主要是由库和传统所决定的。

简单历史

- C语言是B语言发展而来的，B语言是从BCPL发展而来的，BCPL是从FORTRAN发展而来的。
- BCPL和B都支持指针间接方式，所以C也支持。
- C语言还受到了PL/I的影响，还和PDP-11的机器语言有很大的关系。

- 1973.3，第三版的Unix上出现了C语言的编译器
- 1973.11，第四版的Unix（System Four）发布了，这个版本是完全用C语言重新写的。

C的发展与版本 - K&R

经典C（K&R the C）—book→ The C Programming Language, by Brian Kernighan and Dennis Ritchie, 2nd Edition, Prentice Hall

- 标准

1989年ANSI发布了一个标准——ANSI C，1990年ISO接受了ANSI的标准——C89。C的标准在1995年和1999年两次更新——C95和C99。

C是一种工业语言

- 操作系统
- 嵌入式系统
- 驱动程序
- 底层驱动
- 图形引擎、图像处理、声音效果

→

- 开发效率>>学习过程
- 开发效率>>开发乐趣
- 日常~~生活~~应用很少直接用C语言编写
- 学习C的过程主要是写练习代码
- 而非真实软件

编译→运行

- 编辑器
- 编译器

或者IDE（集成开发环境）

1算法的基本逻辑能力(上)

1.第一个C程序

```c
#include <stdio.h>
int main()
{
    printf("Hello World!\n");

        return 0;
}
```

2.做计算

```c
#include <stdio.h>

int main()
{
    printf("23+43=%d\n",23+43);

        return 0;
}
```

四则运算

四则运算	C符号	意义
+	+	加
-	-	减
×	*	乘
÷	/	除
	%	取余
()	()	括号

- %表示取两个数相除以后的余数

3.变量定义

如何输入

1. 输入也在终端窗口中
2. 输入是以行为单位进行的，行的结束标志就是你按下了回车键。在你按下回车之前，你的程序不会读到任何东西

变量



```
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int price = 0;
- 这一行，定义了一个变量。变量的名字是price，类型是int，初始值是0。
- 变量是一个保存数据的地方，当我们需要在程序里保存数据时，比如上面的例子中要记录用户输入的价格，就需要一个变量来保存它。用一个变量保存了数据，它才能参加到后面的计算中，比如计算找零。

变量定义

- 变量定义的一般形式就是：
  - <类型名称> <变量名称>;
- int price;
- int amount;
- int price, amount;

变量的名字

- 变量需要一个名字，变量的名字是一种“标识符”，意思是它是用来识别这个和那个的不同的名字。
- 标识符有标识符的构造规则。基本的原则是：标识符只能由字母、数字和下划线组成，数字不可以出现在第一个位置上，C语言的关键字（有的地方叫它们保留字）不可以用做标识符。

C语言的保留字

auto,break,case,char,const,
continue,default,do,double,
else,enum,extern,float,for,
goto,if,int,long,register,return,
short,signed,sizeof,static,
struct,switch,typedef,union,
unsigned,void,volatile,while,
inline,restrict




```c
#include <stdio.h>
int main()
{
    int price = 0;

    printf("请输入金额（元）：");
    scanf_s("%d", &price);

    int change = 100 - price;
    printf("找您%d元。\n", change);
}
```

有中文，注意切换成符合中文显示的编码！！！

4.变量赋值

赋值和初始化



```
int price = 0;
printf("请输入金额（元）：");
scanf("%d", &price);
int change = 100 - price;
printf("找您%d元。\n", change);
```

- int price = 0;
- 这一行，定义了一个变量。变量的名字是price，类型是int，初始值是0。
- price=0是一个式子，这里的“=”是一个赋值运算符，表示将“=”右边的值赋给左边的变量。



赋值

- 和数学不同，a=b在数学中表示关系，即a和b的值一样；而在程序设计中，a=b表示要求计算机做一个动作：将b的值赋给a。关系是静态的，而动作是动态的。在数学中，a=b和b=a是等价的，而在程序设计中，两者的意思完全相反



初始化

- 当赋值发生在定义变量的时候，就像程序1中的第7行那样，就是变量的初始化。虽然C语言并没有强制要求所有的变量都在定义的地方做初始化，但是所有的变量在第一次被使用（出现在赋值运算符的右边）之前被应该赋值一次
- 如果没有初始化?

```c
#include <stdio.h>

int main()
{
    int i=6;
    int j;
    j = i + 10;
    printf("%d\n", j);

    return 0;
}
```

局部变量在使用前一定要赋初值，不要依赖编译器的“宽容”

变量初始化

- <类型名称> <变量名称> = <初始值>;
  - int price = 0;
  - int amount = 100;
- 组合变量定义的时候，也可以在这个定义中单独给单个变量赋初值，如：
  - int price = 0, amount = 100;

第二个变量



```
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int change = 100 - price;
- 定义了第二个变量change
- 并且做了计算

C99!

ANSI C

- 只能在代码开头的地方定义变量

```
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

C99

```
int price = 0;
int change = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

change = 100 - price;

printf("找您%d元。\n", change);
```

ANSI C

读整数



```
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- scanf("%d", &price);
- 要求scanf这个函数读入下一个整数，读到的结果赋值给变量price
- 小心price前面的&

5.常量vs变量

常量



```
int price = 0;

printf("请输入金额（元）：");
scanf("%d", &price);

int change = 100 - price;

printf("找您%d元。\n", change);
```

- int change = 100 - price;
- 固定不变的数，是常数。直接写在程序里，我们称作直接量（literal）。
- 更好的方式，是定义一个常量：
  - const int AMOUNT = 100;

C99!

const

- const是一个修饰符，加在int的前面，用来给这个变量加上一个const（不变的）的属性。这个const的属性表示这个变量的值一旦初始化，就不能再修改了。
  - int change = AMOUNT - price;
- 如果你试图对常量做修改，把它放在赋值运算符的左边，就会被编译器发现，指出为一个错误。

C99!

```c
#include <stdio.h>
#include <stdlib.h>
int main()
{
    system("chcp 65001 > nul");// 加这一行！强转控制台为 UTF-8
    int a;
```

```
    int b;

    printf("请输入两个整数：");
    scanf_s("%d %d", &a, &b);
    printf("%d + %d = %d\n", a, b, a + b);
}
```

当编码不统一时，有时可以加两行代码偷懒

6.浮点数

带小数点的数值。浮点这个词的本意就是指小数点是浮动的，是计算机内部表达非整数(包含分数和无理数)的一种方式。另一种方式叫做定点数，不过在C语言中你不会遇到定点数。人们借用浮点数这个词来表达所有的带小数点的数。





7.表达式

表达式

- 一个表达式是一系列运算符和算子的组合，用来计算一个值

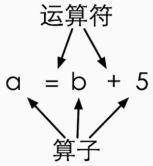
```
amount = x * (1 + 0.033) * (1
+ 0.033) * (1 + 0.033);
total = 57;
count = count.+ 1;
value = (min / 2) * lastValue;
```

运算符

- 运算符（operator）是指进行运算的动作，比如加法运算符“+”，减法运算符“-”。
- 算子（operand）是指参与运算的值，这个值可能是常数，也可能是变量，还可能是一个方法的返回值

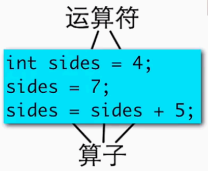
运算符





运算符





计算时间差

- 输入两个时间，每个时间分别输入小时和分钟的值，然后输出两个时间之间的差，也以几小时几分表示

```
int hour1, minute1;
int hour2, minute2;

scanf("%d %d", &hour1, &minute1);
scanf("%d %d", &hour2, &minute2);
```

如果直接分别减，会出现分钟借位的情况：1点40分和2点10分的差?

计算时间差

```
int hour1, minute1;
int hour2, minute2;

scanf("%d %d", &hour1, &minute1);
scanf("%d %d", &hour2, &minute2);

int t1 = hour1 * 60 + minute1;
int t2 = hour2 * 60 + minute2;

int t = t2-t1;

printf("时间差是%d小时%d分。", t/60, t%60);
```

- hour1*60+minute1—>转换为分钟为单位
- t/60—>小时部分；t%60—>分钟

```
#include <stdio.h>
int main()
{
    int hour1, minute1;
    int hour2, minute2;

    scanf_s("%d %d", &hour1, & minute1);
    scanf_s("%d %d", &hour2, & minute2);

    int t1 = hour1 * 60 + minute1;
    int t2 = hour2 * 60 + minute2;

    int t = t2 - t1;

    printf("时间差是 % d小时 % d分。", t / 60, t % 60);

}
```

8.运算符优先级

求平均值

- 写一个程序，输入两个整数，输出它们的平均值

```
int a,b;

scanf("%d %d", &a, &b);

double c = (a+b)/2.0;

printf("%d和%d的平均值=%f\n", a, b, c);
```

运算符优先级

优先级	运算符	运算	结合关系	举例
1	+	单目不变	自右向左	a*+b
1	–	单目取负	自右向左	a*-b
2	*	乘	自左向右	a*b
2	/	除	自左向右	a/b
2	%	取余	自左向右	a%b
3	+	加	自左向右	a+b
3	-	减	自左向右	a-b
4	=	赋值	自右向左	a=b

单目运算符的算子一定在右边，所以它必须自右向左结合。

赋值运算符

- 赋值也是运算，也有结果
- a=6的结果是a被赋予的值，也就是6
- a=b=6 —> a=(b=6)

“嵌入式赋值”

```
int a = 6;
int b;
int c = 1+(b=a);
```

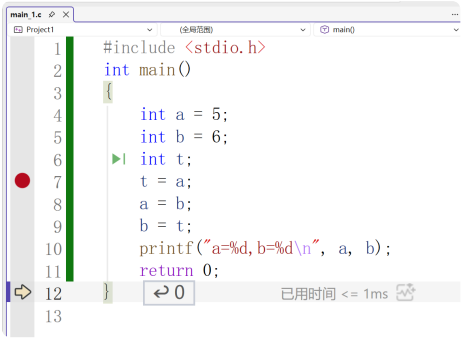
- 不利于阅读
- 容易产生错误

结合关系

- 一般自左向右
- 单目+-和赋值=自右向左

```
result = a = b = 3 + c;
result = 2;
result = (result = result * 2) * 6 * (result = 3 + result);
```

9.交换两个变量




```c
#include <stdio.h>
int main()
{
    int a = 5;
    int b = 6;
    int t;//可在此处加断点逐行查看
    t = a;
    a = b;
    b = t;
    printf("a=%d,b=%d\n", a, b);
    return 0;
}
```

10.复合赋值

复合赋值

- 5个算术运算符，+ - * / %，可以和赋值运算符“=”结合起来，形成复合赋值运算符：“+=”、“-=”、“*=”、“/=”和“%=”
  - total += 5;
  - total = total + 5;
- 注意两个运算符中间不要有空格

复合赋值

- total += (sum+100)/2;
  - total = total + (sum+100)/2;
- total * = sum+12;
  - total = total*(sum+12);
- total /= 12+6;
  - total = total / (12+6);

递增递减运算符

- “++”和“--”是两个很特殊的运算符，它们是单目运算符，这个算子还必须是变量。这两个运算符分别叫做递增和递减运算符，他们的作用就是给这个变量+1或者-1。
  - count++;
  - count += 1;
  - count = count + 1;

前缀后缀



- ++和--可以放在变量的前面，叫做前缀形式，也可以放在变量的后面，叫做后缀形式。
- a++的值是a加1以前的值，而++a的值是加了1以后的值，无论哪个，a自己的值都加了1了。

```c
#include <stdio.h>
int main()
{
    int a;
    a = 10;

    printf("a++=%d\n", a++);
    printf("a=%d\n", a);

    printf("++a=%d\n", ++a);
    printf("a=%d\n",a);

    return 0;
}
```

输出：

a++=10

a=11

++a=12
a=12

前缀后缀

表达式	运算	表达式的值
count++	给count加1	count原来的值
++count	给count加1	count+1以后的值
count--	给count减1	count原来的值
--count	给count减1	count-1以后的值

11.做判断的IF

```
#include <stdio.h>
int main()
{
    int hour1, minute1;
    int hour2, minute2;

    scanf_s("%d %d", &hour1, &minute1);
    scanf_s("%d %d", &hour2, &minute2);

    int ih = hour2 - hour1;
    int im = minute2 - minute1;
    if (im < 0) {
        im = 60 + im;
        ih --;
    }

    printf("时间差是 % d小时 % d分。", ih,im);
```

12.判断条件

条件

- 计算两个值之间的关系，所以叫做关系运算

运算符	意义
= =	相等
!=	不相等
>	大于
>=	大于或等于
<	小于
<=	小于或等于

关系运算的结果

- 当两个值的关系符合关系运算符的预期时，关系运算的结果为整数1，否则为整数0
  - printf("%d\n", 5==3);
  - printf("%d\n", 5>3);
  - printf("%d\n", 5<=3);

```c
#include <stdio.h>
int main()
{//输出结果为0、1、0
    printf("%d\n", 5 == 3);
    printf("%d\n", 5 > 3);
    printf("%d\n", 5 <= 3);

    return 0;
}
```

优先级



- 所有的关系运算符的优先级比算术运算的低，但是比赋值运算的高
  - 7 >= 3 + 4
  - int r = a>0;

优先级

- 判断是否相等的==和!=的优先级比其他的低，而连续的关系运算是从左到右进行的
  - 5 > 3 == 6 > 4
  - 6 > 5 > 4
  - a == b == 6
  - a == b > 0

6>5>4,推出1>4，结果是0

13.找零计算器、流程图、ELSE

找零计算器

- 找零计算器需要用户做两个操作：输入购买的金额，输入支付的票面，而找零计算器则根据用户的输入做出相应的动作：计算并打印找零，或告知用户余额不足以购买。
- 从计算机程序的角度看，这就是意味着程序需要读用户的两个输入，然后进行一些计算和判断，最后输出结果。

```
const int MINOR = 35;

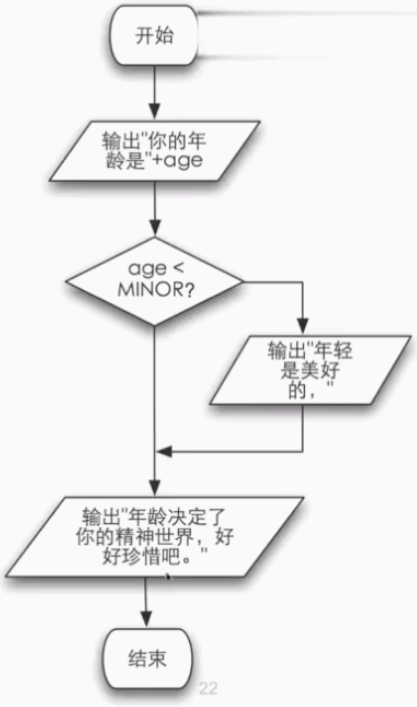
int age = 0;

printf("请输入你的年龄: ");
scanf("%d", &age);

printf("你的年龄是%d岁。\n", age);

if ( age < MINOR ) {
    printf("年轻是美好的，");
}

printf("年龄决定了你的精神世界，好好珍惜吧。\n");
```




```c
#include <stdio.h>
int main()
{
    const int MINOR = 35;

    int age = 0;

    printf("请输入您的年龄: ");
    scanf_s("%d", &age);
```

```
    printf("您的年龄是%d岁。\n", age);

    if (age < MINOR) {
        printf("年轻是美好的，");
    }

    printf("年龄决定了你的精神世界，好好珍惜吧。\n");
}
```

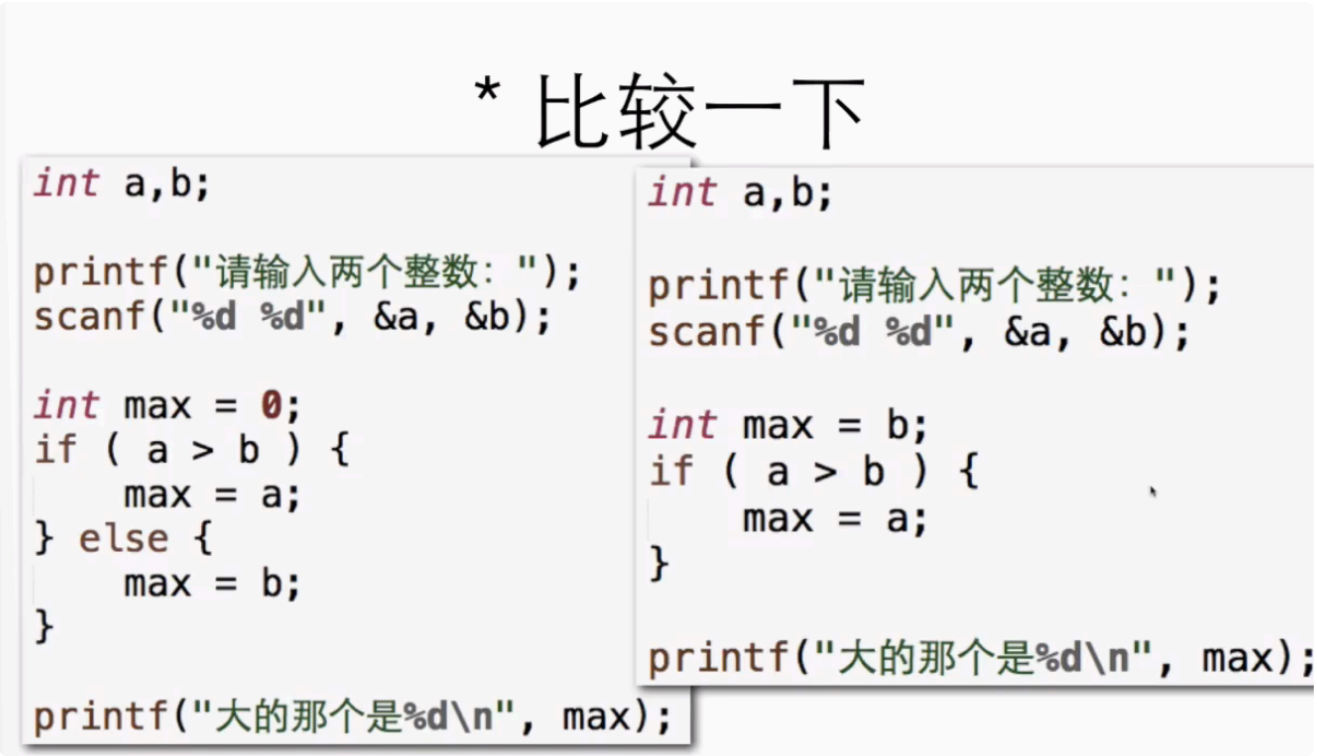
else

```
//  初始化
int price = 0;
int bill = 0;
//  读入金额和票面
printf("请输入金额: ");
scanf("%d", &price);
printf("请输入票面: ");
scanf("%d", &bill);
//  计算找零
if ( bill >= price ) {
    printf("应该找您: %d\n", bill - price);
} else {
    printf("你的钱不够\n");
}
```

- else = 否则的话

```
#include <stdio.h>
int main()
{
    //初始化
    int price = 0;
    int bill = 0;
    //读入金额和票面
    printf("请输入金额:");
    scanf_s("%d", &price);
    printf("请输入票面:");
    scanf_s("%d", &bill);
    //计算找零
    if (bill >= price) {
        printf("应该找您：%d\n", bill - price);
    }
    else {
```

```
        printf("你的钱不够\n");
    }
}
```



1.更易读

```
#include <stdio.h>
int main()
{
    int a, b;

    printf("请输入两个整数: ");
    scanf_s("%d %d", &a, &b);

    int max = 0;
    if (a > b) {
        max = a;
    }
    else {
        max = b;
    }

    printf("大的那个是%d\n", max);
}
```

2.更简洁

```c
#include <stdio.h>
int main()
{
    int a, b;

    printf("请输入两个整数：");
    scanf_s("%d %d", &a, &b);

    int max = b;
    if (a > b) {
        max = a;
    }

    printf("大的那个是%d\n", max);
}
```

14回探IF



if语句

- 一个基本的if语句由一个关键字if开头，跟上在括号里的一个表示条件的逻辑表达式，然后是一对大括号“{}”之间的若干条语句。如果表示条件的逻辑表达式的结果不是零，那么就执行后面跟着的这对大括号中的语句，否则就跳过这些语句不执行，而继续下面的其他语句。

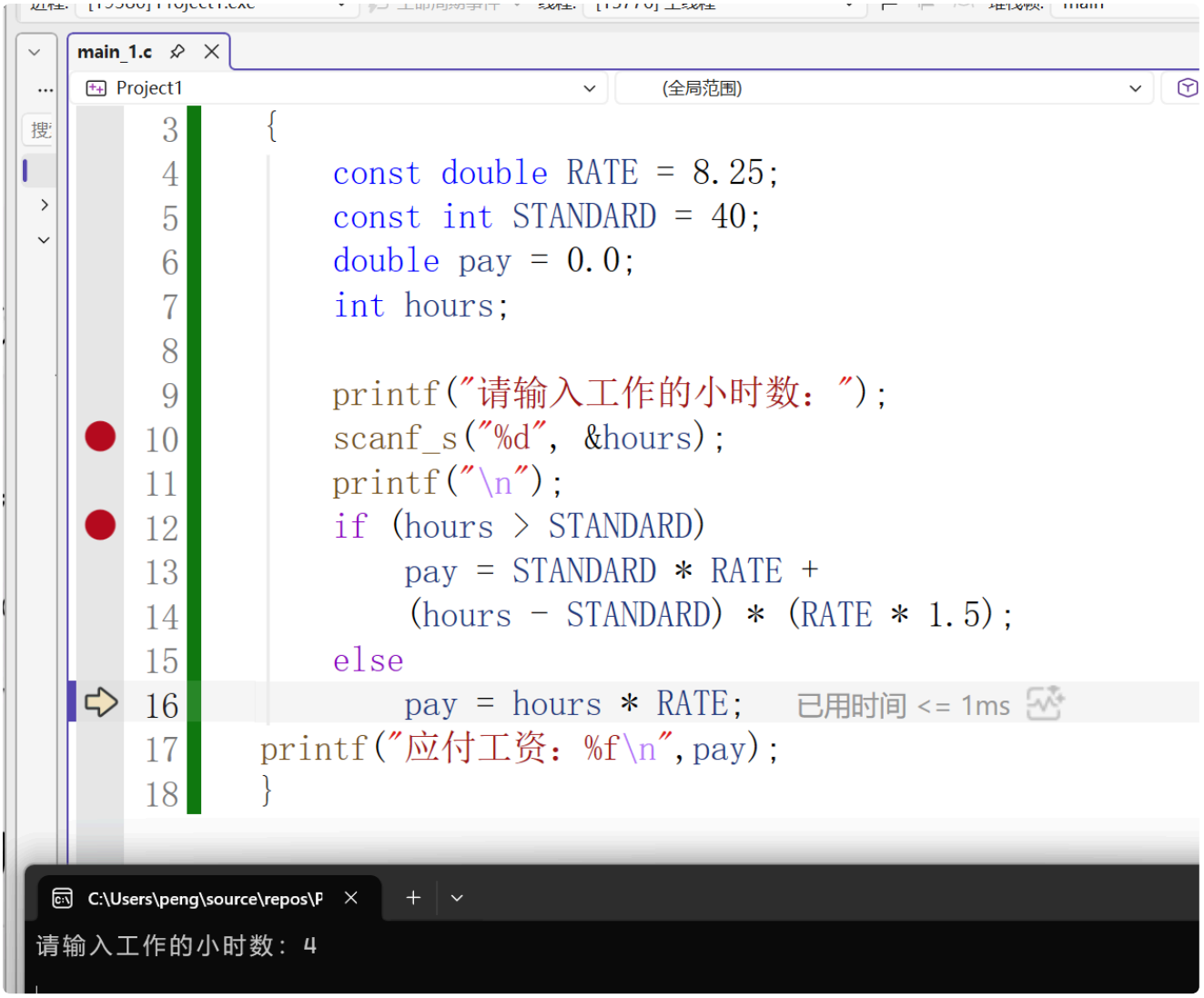
```
if ( total > amount )
        total += amount+10;
```

if语句

- 一个基本的if语句由一个关键字if
上在括号里的一个表示条件的逻
式，然后是一对大括号“{}”之间
语句。如果表示条件的逻辑表达
果不是零，那么就执行后面跟着
大括号中的语句，否则就跳过这
不执行，而继续下面的其他语句

```
if ( total > amount )
        total += amount+10;
```

if语句这一行结束的时候并没有表示语句结束的“;”，而后面的赋值语句写在if的下一行，并且缩进了，在这一行结束的时候有一个表示语句结束的“;”。这表明这条赋值语句是if语句的一部分，if语句拥有和控制这条赋值语句，决定它是否要被执行。




```c
#include <stdio.h>//计算薪水
int main()
{
    const double RATE = 8.25;
    const int STANDARD = 40;
    double pay = 0.0;
    int hours;

    printf("请输入工作的小时数：");
    scanf_s("%d", &hours);
    printf("\n");
    if (hours > STANDARD)
        pay = STANDARD * RATE +
        (hours - STANDARD) * (RATE * 1.5);
    else
        pay = hours * RATE;
printf("应付工资：%f\n",pay);
}
```

判断成绩



```
const int PASS=60;
int score;

printf("请输入成绩: ");
scanf("%d", &score);

printf("你输入的成绩是%d.\n", score);
if ( score < PASS )
    printf("很遗憾，这个成绩没有及格。");
else
    printf("祝贺你，这个成绩及格了。");
printf("再见\n");
```

```c
#include <stdio.h>
int main()
{
    const int PASS = 60;
    int score;

    printf("请输入成绩:");
    scanf_s("%d", &score);

    printf("你输入的成绩是%d.\n", score);
    if (score < PASS)
        printf("很遗憾，这个成绩没有及格。");
    else {
        printf("祝贺你，这个成绩及格了。");
        printf("再见\n");
    }
    return 0;
}
```

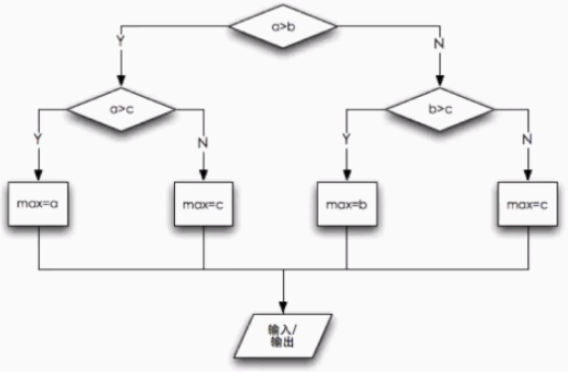
15.嵌套的IF-ELSE

```
int a,b,c;
scanf("%d %d %d", &a, &b, &c);

int max = 0;

if ( a>b ) {
    if ( a>c ) {
        max = a;
    } else {
        max = c;
    }
} else {
    if ( b>c ) {
        max = b;
    } else {
        max = c;
    }
}

printf("The max is %d\n", max);
```




```c
#include <stdio.h>
int main()
{
    int a, b, c;
    scanf_s("%d %d %d", &a, &b, &c);

    int max = 0;

    if (a > b) {
        if (a > c) {
            max = a;
        }
        else {
            max = c;
        }
    }
    else {
        if (b > c) {
            max = b;
        }
        else {
            max = c;
        }
    }
    printf("The max is %d\n", max);
```

```
    return 0;
}
```

```
int a,b,c;
scanf("%d %d %d", &a, &b, &c);

int max = 0;

if ( a>b ) {
    if ( a>c ) {
        max = a;
    } else {
        max = c;
    }
} else {
    if ( b>c ) {
        max = b;
    } else {
        max = c;
    }
}

printf("The max is %d\n", max);
```

```
int a,b,c;
scanf("%d %d %d", &a, &b, &c);

int max = 0;

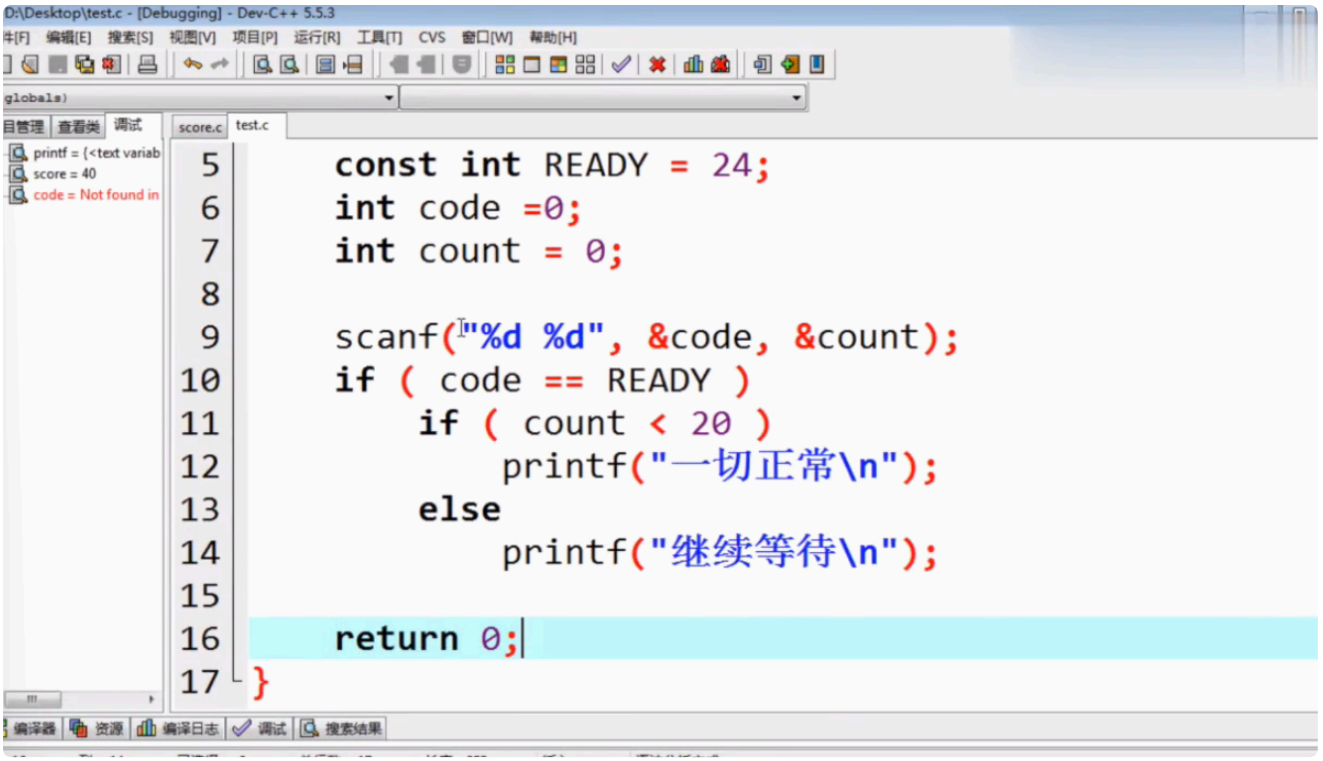
if ( a>b )
    if ( a>c )
        max = a;
    else
        max = c;
else
    if ( b>c )
        max = b;
    else
        max = c;

printf("The max is %d\n", max);
```

嵌套的判断

- 当if的条件满足或者不满足的时候要执行的语句也可以是一条if或if-else语句，这就是嵌套的if语句

```
if ( code == READY )
   if ( count <20 )
      printf("一切正常\n");
   else
      printf("继续等待\n");
```



他喵喵喵的竟然坑我，没有花括号。

else的匹配

- else总是和最近的那个if匹配

```
if ( code == READY ) {
    if ( count <20 )
        printf("一切正常\n");
} else
    printf("继续等待\n");
```



验证

```
#include <stdio.h>
int main()
{
    const int READY = 24;
    int code = 0;
```

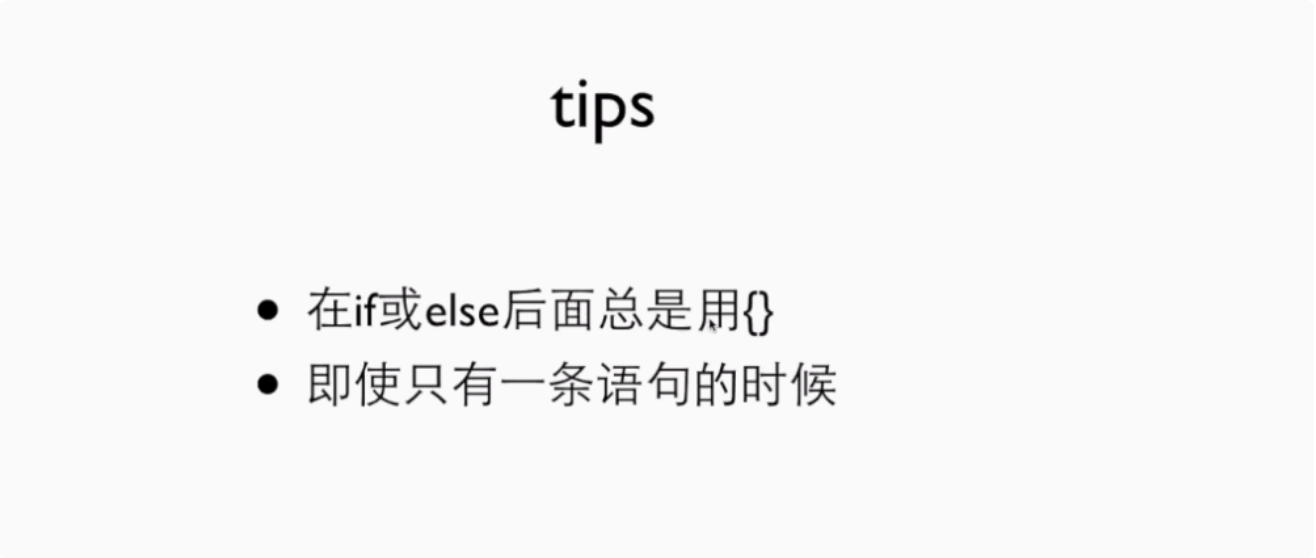
```
    int count = 0;

    scanf_s("%d %d", &code, &count);
    if (code == READY) {
        if (count < 20) {
            printf("一切正常\n");
        }
        else {
            printf("继续等待\n");
        }
    }else {
        printf("继续等待\n");
}
    return 0;
}
```

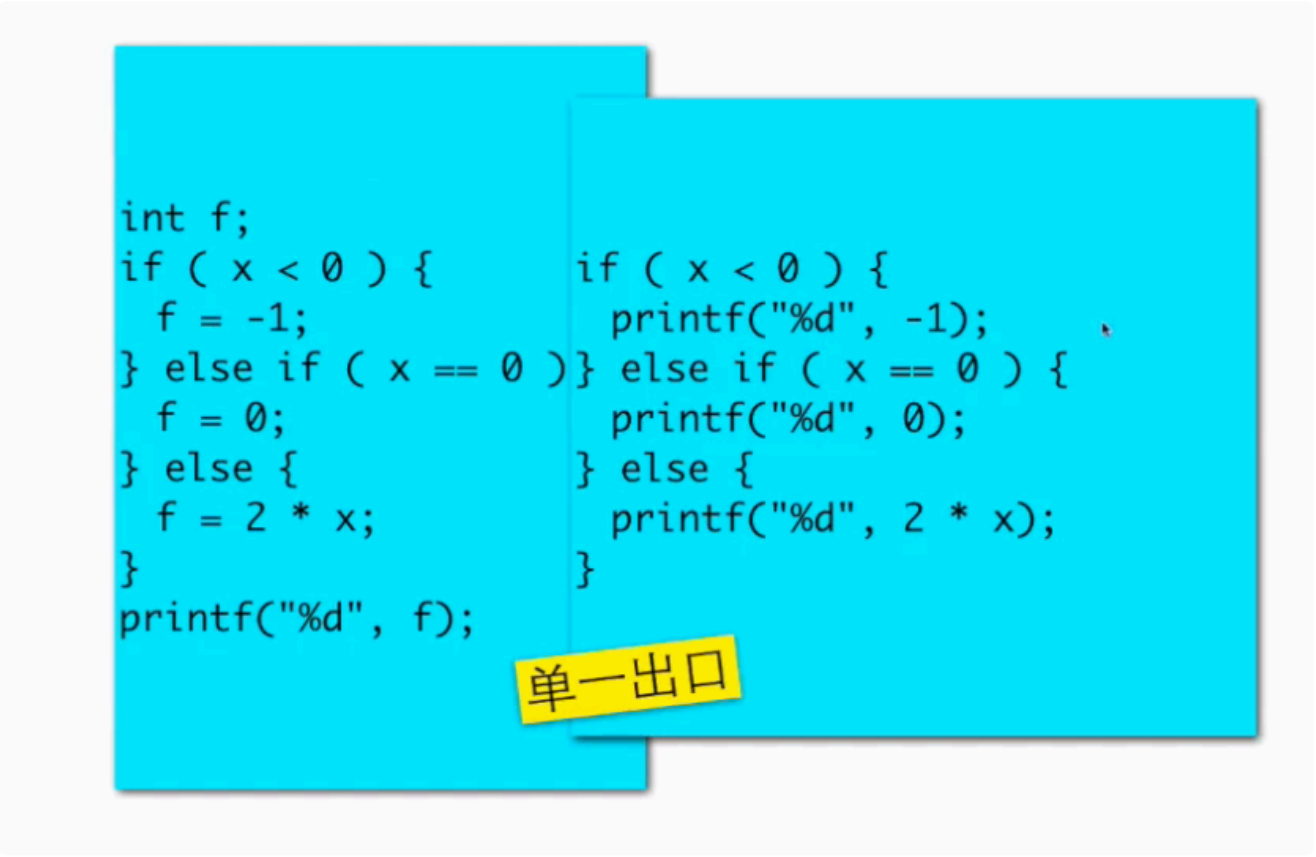
简洁版：

```
#include <stdio.h>
int main()
{
    const int READY = 24;
    int code = 0;
    int count = 0;

    scanf_s("%d %d", &code, &count);
    if (code == READY && count < 20){
    printf("一切正常\n");
    } else {
        printf("继续等待\n");
}
    return 0;
}
```



16.级联的IF-ELSE IF

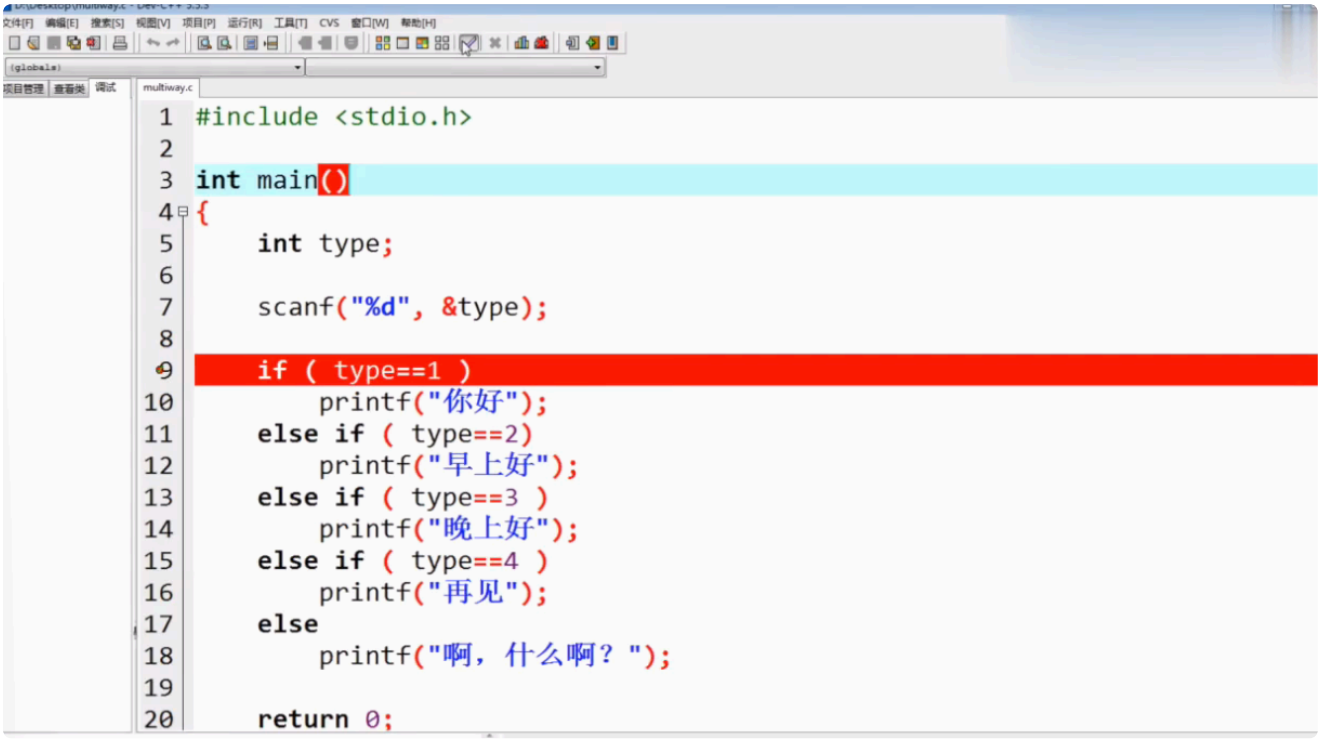


17.多路分支

switch-case

```
switch ( 控制表达式 ) {
case 常量:
   语句
   ……
case 常量:
   语句
   ……
default:
   语句
   ……
}
```

- 控制表达式只能是整数型的结果
- 常量可以是常数，也可以是常数计算的表达式



```c
#include <stdio.h>

int main()
{
    int type;

    scanf("%d", &type);

    if ( type==1 )
        printf("你好");
    else if ( type==2)
        printf("早上好");
    else if ( type==3 )
        printf("晚上好");
    else if ( type==4 )
        printf("再见");
    else
        printf("啊，什么啊？ ");

    return 0;
```

switch.c

```c
#include <stdio.h>

int main()
{
    const int MRN = 2;
    int type;

    scanf("%d", &type);

    switch ( type ) {
    case 1:
        printf("你好");
        break;
    case MRN:
        printf("早上好");
        break;
    case 3:
        printf("晚上好");
        break;
    case 4:
        printf("再见");
        break;
    default:
        printf("啊，什么啊？ ");
        break;
```

[Finished in 0.1s]

C99 ONLY

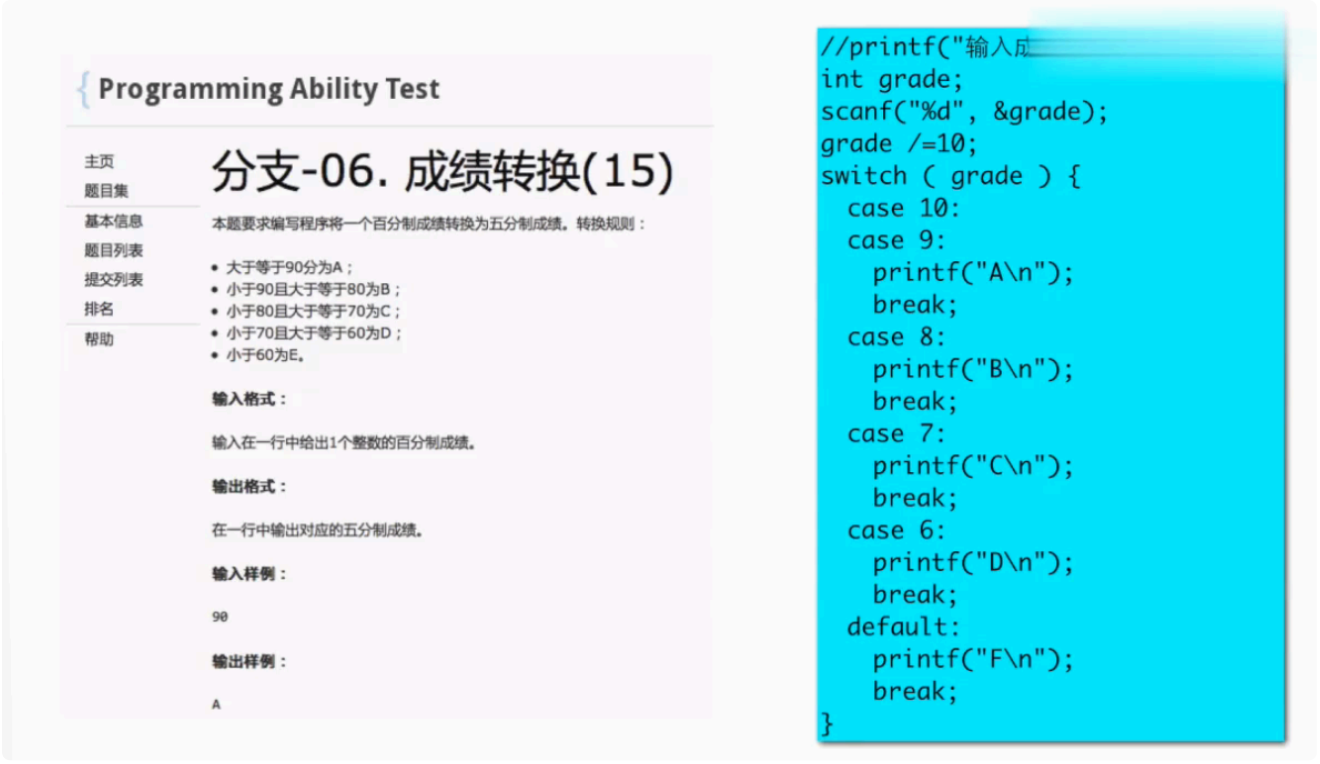
```c
switch ( type )
{
case 1:
case 2:
   printf("你好\n");
   break;
case 3:
   printf("晚上好\n");
case 4:
   printf("再见\n");
   break;
default:
   printf("啊，什么啊？\n");
   break;
}
```

break

- switch语句可以看作是一种基于计算的跳转，计算控制表达式的值后，程序会跳转到相匹配的case（分支标号）处。分支标号只是说明switch内部位置的路标，在执行完分支中的最后一条语句后，如果后面没有break，就会顺序执行到下面的case里去，直到遇到一个break，或者switch结束为止。

case不是划分的标志。它只是一个路牌，它不能阻止程序继续往后运行。





注：这段代码不符合“单一出口”的原则，因为我们还没学过字符或字符串数据的处理

```
printf("请输入月份: ");
int month;
scanf("%d", &month);
switch ( month ) {
      case 1: printf("January\n"); break;
      case 2: printf("February\n"); break;
      case 3: printf("March\n"); break;
      case 4: printf("April\n"); break;
      case 5: printf("May\n"); break;
      case 6: printf("June\n"); break;
      case 7: printf("July\n"); break;
      case 8: printf("August\n"); break;
      case 9: printf("September\n"); break;
      case 10:printf("October\n"); break;
      case 11:printf("November\n"); break;
      case 12:printf("December\n"); break;
}
```

今后可以用数组来做

try

$$f(x) = \begin{cases} -1; & x<0 \\ 0; & x=0 \\ 2x; & x>0 \end{cases}$$

- 这个分段函数能否用switch-case?

能，但但完全没必要。因为 `switch-case` 在 C 语言里只能进行“等值判断”（离散值），不能进行“范围判断"。

如果非要强行用 switch-case，可以映射成一个具体的离散数字。利用 C 语言里布尔值（真为1，假为0）的特性，我们可以写一个数学式子：`(x > 0) - (x < 0)`。

```
switch ( (x > 0) - (x < 0) ) {
   case -1:
```

```
        printf("f(x) = -1\n");
        break;
    case 0:
        printf("f(x) = 0\n");
        break;
    case 1:
        printf("f(x) = 2 * x\n");
        break;
}
```

凡是遇到“范围判断”（大于、小于、区间），一律用 `if-else if-else`。